© 2026 ANSYS, Inc. or affiliated companies
Unauthorized use, distribution, or duplication prohibited.

ansys-pyetk

ANSYS, Inc.
Southpointe
2600 Ansys Drive
Canonsburg, PA 15317
ansysinfo@ansys.com
http://www.ansys.com
(T) 724-746-3304
(F) 724-514-9494

Avr. 17, 2026

ANSYS, Inc. and ANSYS Europe, Ltd. are UL registered ISO 9001:2015 companies.

CONTENTS

Useful links: *Installation* | Source repository | Issues

The Electronic Transformer ToolKit (PyETK) provides a solution for designing electronic transformers. PyETK uses PyAEDT methods and best practices to develop electronic transformers. It improves efficiency, consistency, and collaboration when creating and developing AEDT magnetic component models.

Getting started Learn how to install PyETK and understand transformer dimensions in two main build topologies, the wound and planar build types.

Getting started User guide Learn how to use the PyETK common framework and the **Transformer Builder** tab to design and model electronic transformers.

User guide API reference Understand the PyETK APIs.

API reference Examples Explore examples that show how to use PyETK.

Examples

CHAPTER

ONE

GETTING STARTED

This section explains how to install PyETK. It also describes transformer dimensions in two main build topologies, the wound and planar build types.

Installation Learn how to install PyETK.

Installation Dimensions Understand transformer dimensions in wound and planar build types.

Dimensions

1.1 Installation

You can install PyETK using an installer, using Python, or in developer mode. Choose the method that best suits your needs.

1.2 Install PyETK using an installer

The repositorys Releases page provides for downloading PyETK assets. For the latest PyETK release, use one of these links to download the installer for your operating system:

- Windows installer
- Ubuntu 22.04 installer
- Ubuntu 24.04 installer

Windows

Follow these steps to use the Windows installer to install PyETK in user mode:

1. Download the Windows installer. The file should be named `PyETK-Installer-Windows.exe`.
2. Run the installer.
3. Search for `PyETK` and run it.

The PyETK UI opens.

Linux

On Linux, supported operating systems are Ubuntu 22.04 and 24.04.

Follow these steps to use a Linux installer to install PyETK in user mode:

1. Run the following commands to update the `apt-get` repository and install these packages with **sudo** privileges: `wget`, `gnome`, `libffi-dev`, `libssl-dev`, `libsqlite3-dev`, `libxcb-xinerama0`, and `build-essential`:

```
sudo apt-get update -y
sudo apt-get install wget gnome libffi-dev libssl-dev libsqlite3-dev libxcb-
↪xinerama0 build-essential -y
```

2. Install the `zlib` package:

```
wget https://zlib.net/current/zlib.tar.gz
tar xvzf zlib.tar.gz
cd zlib-*
make clean
./configure
make
sudo make install
```

3. Install the Toolkit Template:
   1. Download the latest `Toolkit-Template_*.zip` file from the repositorys Release page.
   2. Run this command on the terminal:

      ```
      unzip Toolkit-Template_*.zip
      ./installer.sh
      ```

   3. Search for `PyETK` and run it.

The PyETK UI opens.

To uninstall PyETK, follow these steps.

1. Select **File > Uninstall**.
2. Click **Uninstall**.

1.3 Install PyETK using Python

You can use Python to install from PyPI either both the PyETK backend and UI methods or only the backend methods.

To install both the backend and UI methods:

```
pip install ansys-pyetk[all]
```

To install only the backend methods:

```
pip install ansys-pyetk
```

1.4 Install PyETK in developer mode

Installing PyETK in developer mode allows you to modify the source and enhance it.

> **Note**
>
> Before contributing to the project, see the PyAnsys developers guide.

You can install PyETK in developer mode with only these few lines of code:

```
git clone https://github.com/ansys/ansys-pyetk
cd ansys-pyetk
python -m pip install -U pip uv
uv venv
uv pip install -e .
```

To run PyEDT, use this command:

```
uv run run_toolkit
```

Work environment setup

To set up your work environment for development, follow these steps:

1. Create a fresh-clean Python environment and activate it. For more information, see the venv Python documentation.

   ```
   # Create a virtual environment
   python -m uv venv .venv

   # Activate it in a POSIX system
   source .venv/bin/activate

   # Activate it in Windows CMD environment
   .venv\Scripts\activate.bat

   # Activate it in Windows Powershell
   .venv\Scripts\Activate.ps1
   ```

2. Make sure you have the latest version of pip:

   ```
   python -m pip install -U pip uv
   ```

3. Install the project in editable mode:

   ```
   python -m uv pip install -e .[tests,doc]
   ```

4. Verify your development installation by running the tests:

   ```
   uv run pytest tests -v
   ```

1.4.1 Run style checks and tests

This project uses pre-commit to run style checks and tests.

Install `pre-commit`:

```
uv pip install pre-commit
uv run pre-commit install
```

After each commit, `pre-commit` runs to ensure your changes follow project style guidelines:

```
git commit -am 'fix style'
isort....................................................................Passed
black....................................................................Passed
```

(continues on next page)

(continued from previous page)

```
blacken-docs.............................................................Passed
flake8...................................................................Passed
codespell................................................................Passed
pydocstyle...............................................................Passed
check for merge conflicts................................................Passed
debug statements (python)................................................Passed
check yaml...............................................................Passed
trim trailing whitespace.................................................Passed
Validate GitHub Workflows................................................Passed
```

If you need to run `pre-commit` again on all files and not just staged files, run it with the `--all-files` option:

```
uv run pre-commit run --all-files
```

1.4.2 Deploy a local build

You can deploy PyETK as a *frozen* application using PyInstaller:

```
uv pip install -e .[freeze]
uv run pyinstaller frozen.spec
```

This generates app files at `dist/ansys_pyetk/Electronic Transformer Toolkit.exe`. The `Electronic Transformer Toolkit.exe` is a standalone application, without the need to install Python or any dependencies.

It is also possible to deploy an Windows installer using `NSIS`.

For more information see: Distributing Toolkits

1.4.3 Build documentation

To build the documentation, run the usual rules provided in the Sphinx Makefile:

```
uv pip install -e .[doc]
uv run make -C doc/ html

# subsequently open the documentation with (under Linux):
<your_browser_name> doc/html/index.html
```

1.5 Dimensions

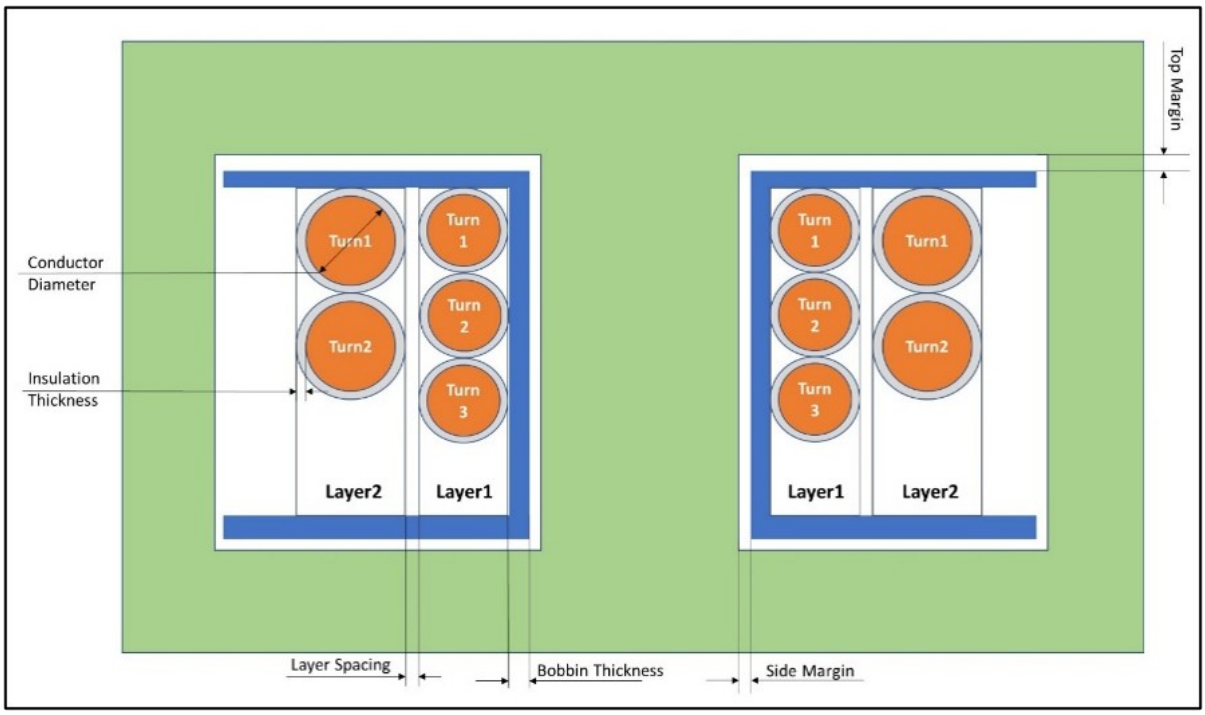
This page explains the key dimensions used throughout PyETK. Understanding these dimensions streamlines your workflow and ensures you use every feature to its full potential.

The transformer dimensions, from core to turn distances, fall into two main build topologies:

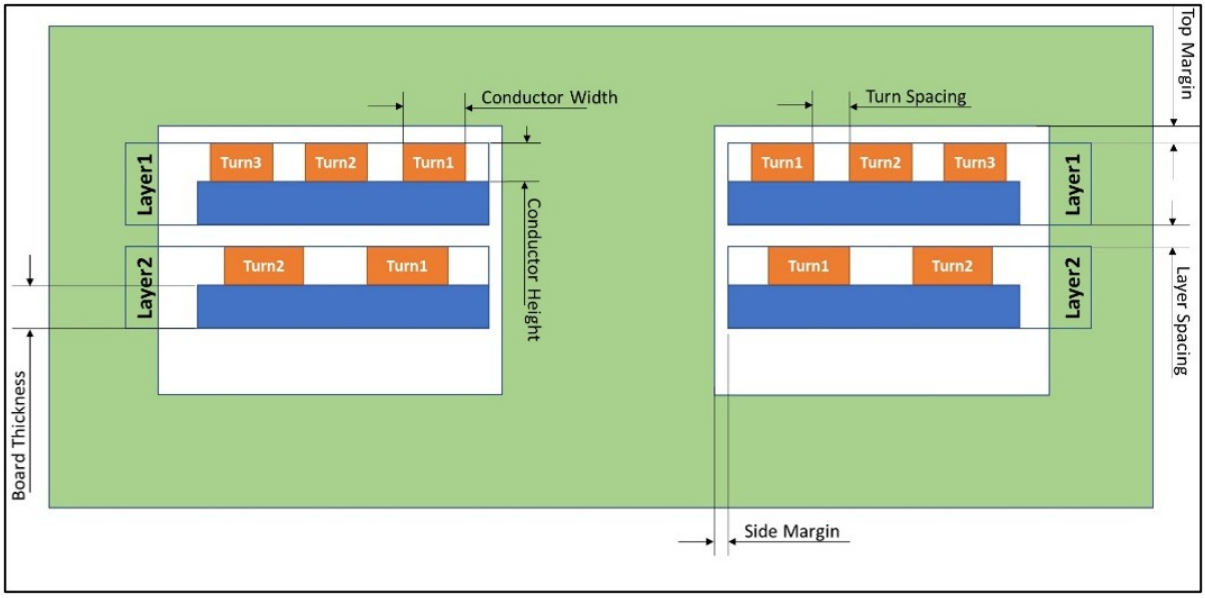
- Wound build type
- Planar build type

The main difference between the two is that the planar transformers winding is embedded in the PCB as traces, while the wound winding wraps around a leg of the magnetic core.

1.5.1 Wound build type



1.5.2 Planar build type



CHAPTER

TWO

USER GUIDE

This section describes how to use the PyETK common framework and the **Transformer Builder** tab to design and model electronic transformers. It assumes that PyETK has already been installed and launched. For more information, see *Installation*.

Common framework Learn how to navigate the PyETK common framework.

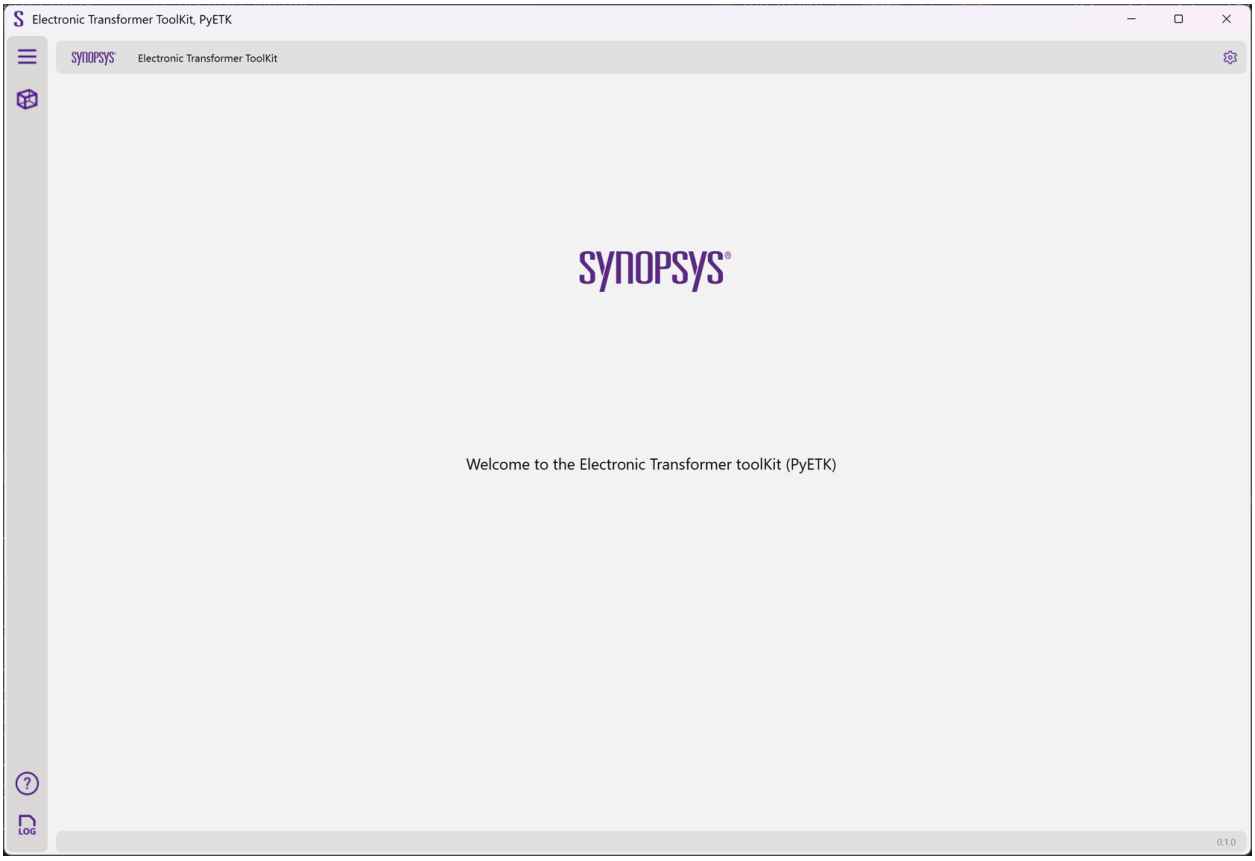
Common framework

Transformer Builder tab Learn how to use the **Transformer Builder** tab to specify transformer settings.

Transformer Builder tab

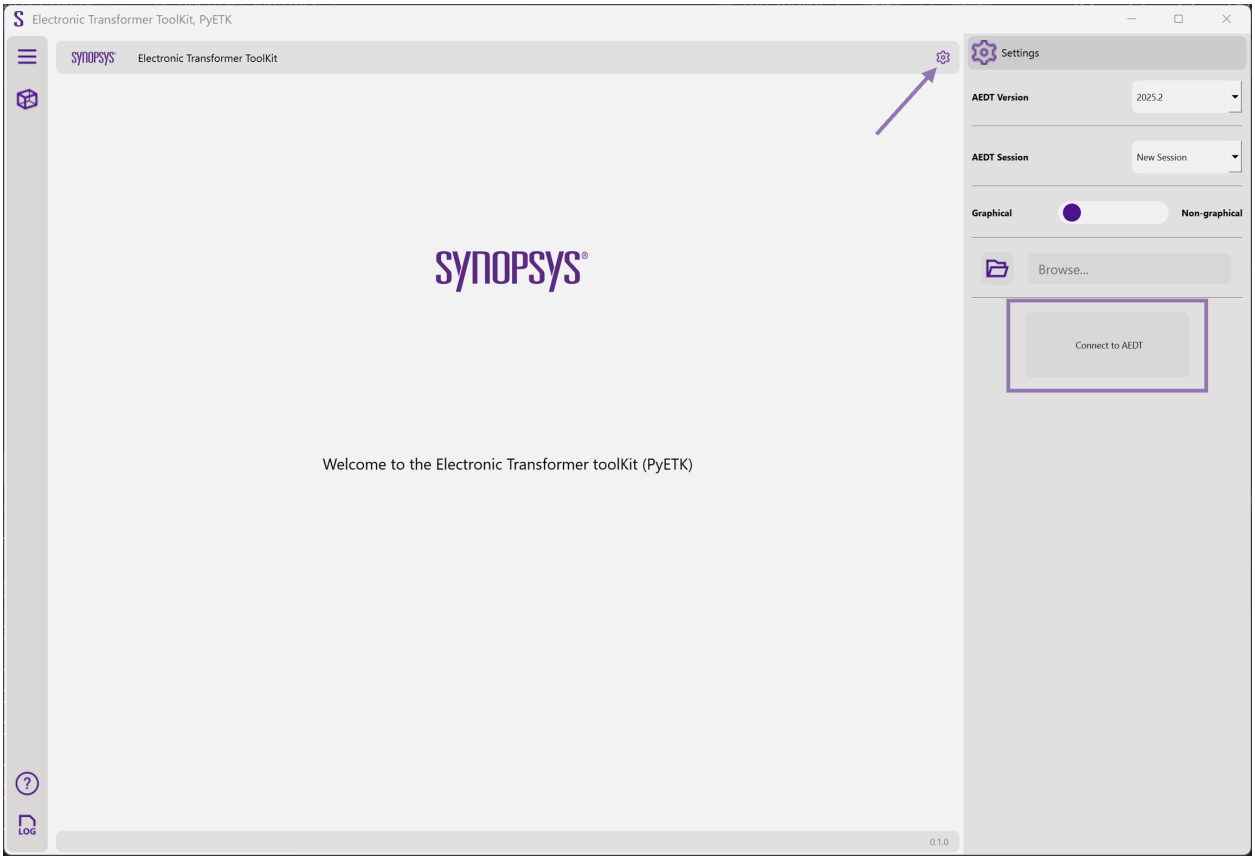
2.1 Common framework

This page explains how to use the PyETK common framework, which is shared across all AEDT toolkits. It takes you through the UI, from the initial **Welcome** page to configuring settings and viewing logs and online help.

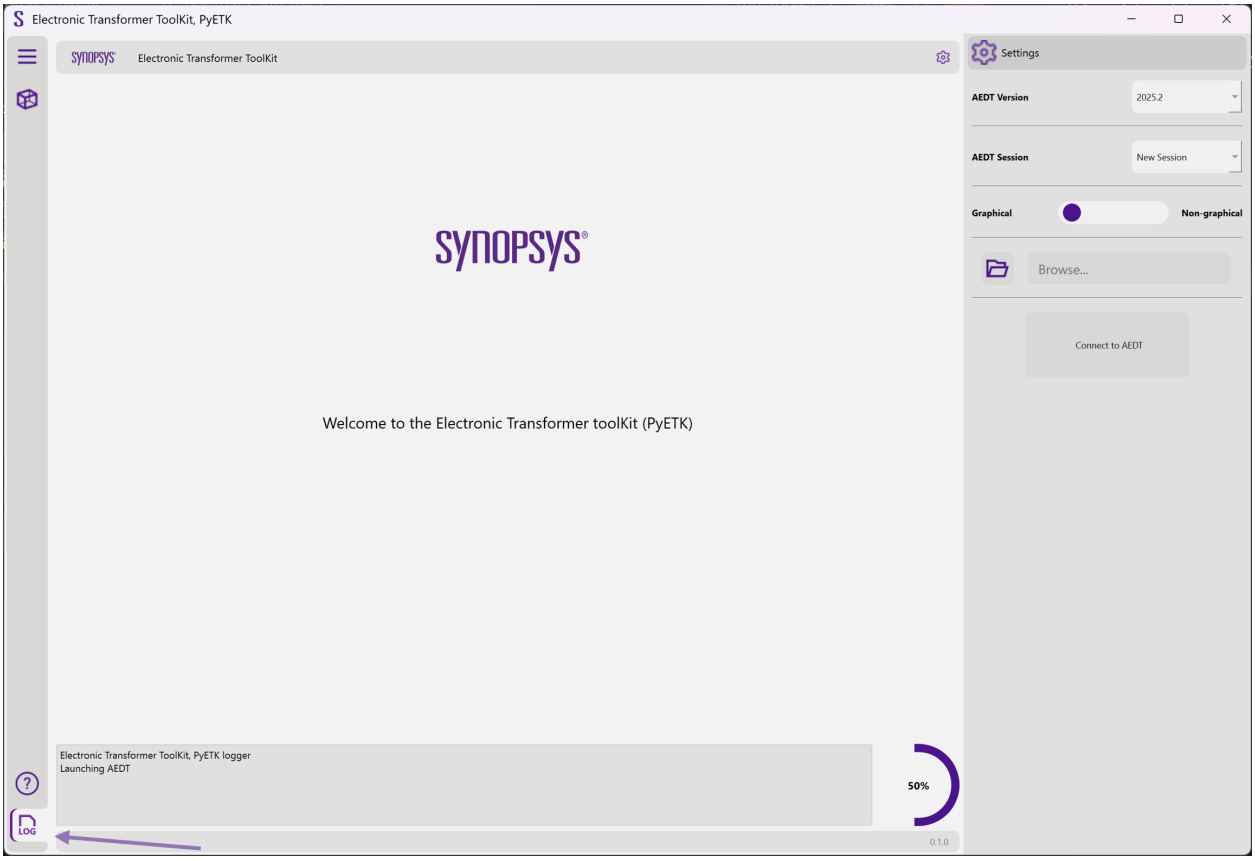


In the UI **Settings**, specify settings for either creating an AEDT session or connecting to an existing AEDT session.

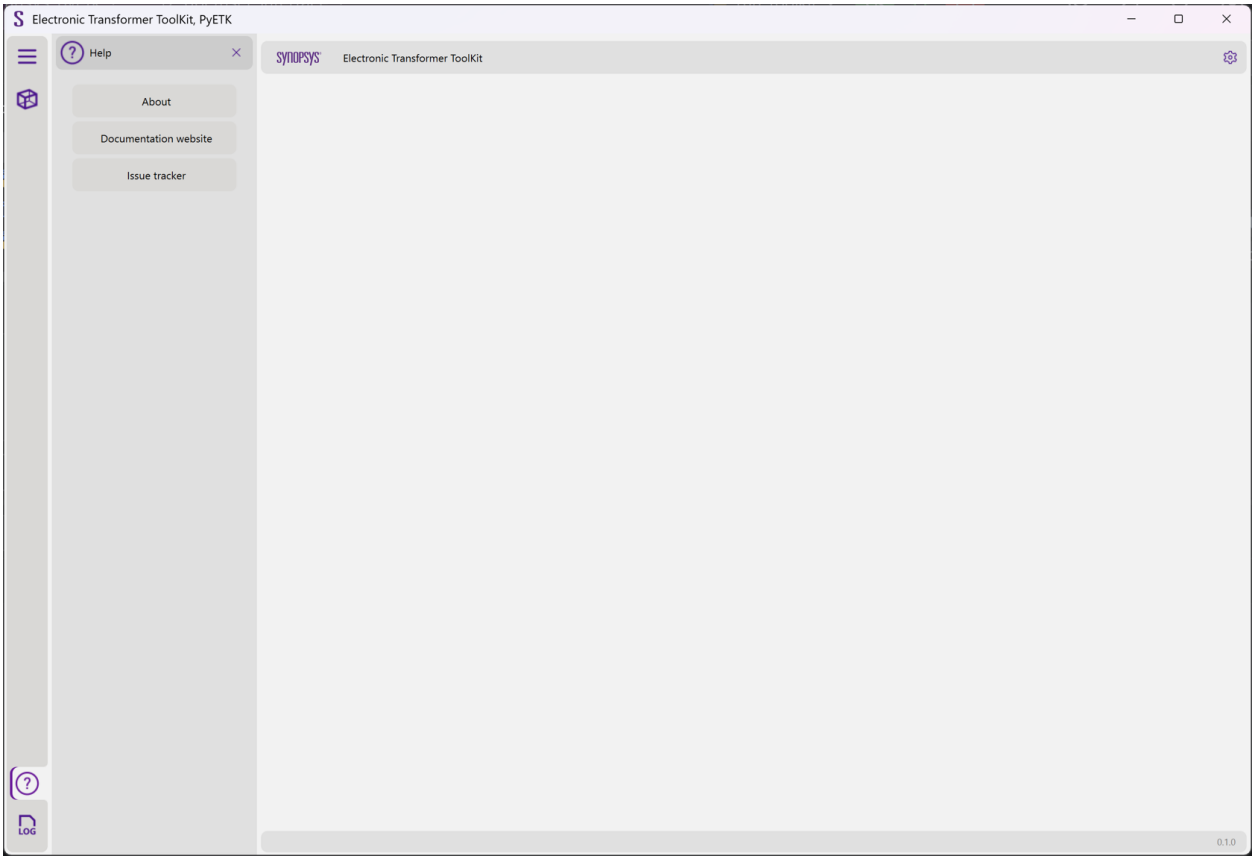
> **Note**
>
> If the PyETK UI is launched from AEDT, the **Settings** icon does not appear because the toolkit is directly connected to the specific AEDT session.



In the lower left corner, click the **Log** icon to view general information about the toolkits operations, such as the status of the connection to AEDT, the creation of geometries, and any errors that might have occurred during the process.

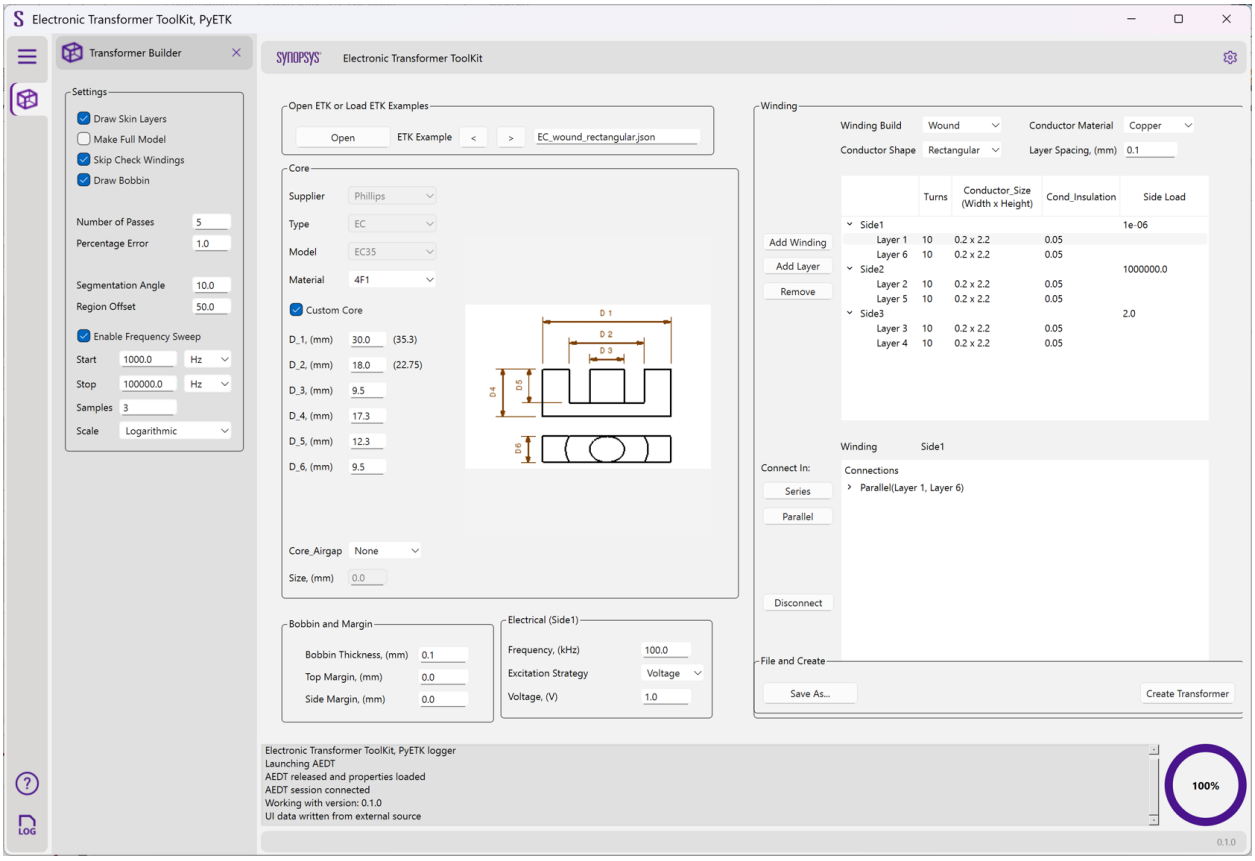


In the lower left corner, click the **Help** icon to access information about your PyETK installation, PyETK documentation, and issues tracked in the GitHub repository.



2.2 Transformer Builder tab

In the UIs upper left corner, click the cube icon to display the **Transformer Builder** tab.



In this tab, specify transformer information:

- Core type
- Core dimensions
- Bobbin type
- Bobbin dimensions
- Winding data
- Excitation data

Use the **Settings** pane on the left side of the **Transformer Builder** tab to specify Maxwell analysis settings:

- Number of passes
- Percentage error
- Maximum number of passes
- Frequency sweeps

CHAPTER

THREE

API REFERENCE

The PyETK API contains the `ToolkitBackend` class, which provides methods for creating the whole model or specific geometries, such as the core, winding, or bobbin. These methods help you to visualize those parts and create core types, bobbins, excitations, circuits, and ultimately assemble an electronic transformer model. You use the PyETK API at the toolkit level.

The PyAEDT Common Toolkit provides common methods to create an AEDT session or connect to an existing AEDT session.

For information on using the PyETK API, see *Create a transformer model in AEDT using the PyETK API*.

class `ansys.aedt.toolkits.electronic_transformer.backend.api.`**`ToolkitBackend`**

Bases: `AEDTCommon`

Control the toolkit workflow.

This class provides methods to connect to a selected design and create the model or the parts of the transformer.

Examples

```
>>> from ansys.aedt.toolkits.electronic_transformer.backend.api import␣
↪ToolkitBackend
>>> toolkit_api = ToolkitBackend()
>>> toolkit_api.launch_aedt()
>>> toolkit_api.wait_to_be_idle()
>>> toolkit_api.create_core_geometry()
```

`create_bobbin_geometry`(*frontend_properties=None*)

Create the bobbin geometry.

Parameters

frontend_properties
[`dict`, default: `None`] Frontend properties.

Returns

`ansys.aedt.toolkits.electronic_transformer.backend.workflows.bobbin.Bobbin`
Bobbin object.

`create_core_geometry`(*frontend_properties=None*)

Create the core geometry.

Parameters

frontend_properties
[`dict`, default: `None`] Frontend properties.

Returns

ansys.aedt.toolkits.electronic_transformer.backend.workflows.core.Core
Core object.

create_model(*frontend_properties=None*)

Create the complete model.

Parameters

frontend_properties
[dict, default: None] Frontend properties.

Returns

ansys.aedt.toolkits.electronic_transformer.backend.workflows.etk.ETK
ETK object.

create_winding_geometry(*frontend_properties=None*)

Create the winding geometry.

Parameters

frontend_properties
[dict, default: None] Frontend properties.

Returns

ansys.aedt.toolkits.electronic_transformer.backend.workflows.winding.Winding
Winding object.

load_properties_from_json(*file_path*)

Load properties from a JSON file.

Parameters

file_path
[str] Path to the JSON file.

validate_json(*data*)

Validate input data from a JSON file.

Parameters

data
[pathlib.Path] Path to the input data in JSON format.

validate_model(*frontend_properties=None*)

Validate the transformer model configuration.

Parameters

frontend_properties
[dict, optional] Frontend properties. The default is None.

Returns

tuple
Tuple containing validation status (bool) and list of error messages.

CHAPTER

FOUR

EXAMPLES

This section provides examples that show how to use PyETK.

Create a transformer model in AEDT using the PyETK API Use the PyETK API to create a class instance, set common properties, and define where to read transformer properties.

Create a transformer model in AEDT using the PyETK API Create a transformer model in AEDT using the PyETK UI Use the fields and tools in the PyETK UI to specify settings for designing a planar transformer.

Create a transformer model in AEDT using the PyETK UI Migrate ACT JSON files to PyETK JSON files Use the PyETK UI to migrate an ACT JSON file to a PyETK JSON file.

Use the PyETK UI to migrate an ACT JSON file to a PyETK JSON file

Copyright (C) 2023 - 2026 ANSYS, Inc. and/or its affiliates. SPDX-License-Identifier: Apache-2.0

Licensed under the Apache License, Version 2.0 (the License); you may not use this file except in compliance with the License. You may obtain a copy of the License at

```
http://www.apache.org/licenses/LICENSE-2.0
```

Unless required by applicable law or agreed to in writing, software distributed under the License is distributed on an AS IS BASIS, WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied. See the License for the specific language governing permissions and limitations under the License.

4.1 Create a transformer model in AEDT using the PyETK API

This example shows how to use the `ToolkitBackend` class in the PyEDT API to create a transformer model in AEDT. More specifically, it does the following:

- Initializes the `ToolkitBackend` class.
- Sets common properties like `non_graphical` and `aedt_version`.
- Defines the path to the JSON file with the transformer properties.
- Reads the transformer properties from the JSON file.
- Launches AEDT.
- Creates the transformer model.

[1]:
```
# ## Perform required imports
from pathlib import Path
import sys
import tempfile
```

[2]:
```
from ansys.aedt.core import generate_unique_project_name
from ansys.aedt.toolkits.electronic_transformer.backend.api import ToolkitBackend
```

[3]:
```
# ## Create backend instance
toolkit_api = ToolkitBackend()
```

[4]:
```
# ## Set common properties
toolkit_api.properties.non_graphical = False
toolkit_api.properties.aedt_version = "2025.2"
```

[5]:
```
# ## Define the JSON path
json_file_name = "RM_wound_circular.json"
temp_dir = tempfile.TemporaryDirectory(suffix="_ansys")
project_name = generate_unique_project_name(root_name=temp_dir.name, project_name="ETK_
↪example")
try:
    # Works in standard .py scripts
    json_path = Path(__file__).resolve().parents[3]
except NameError:
    # Fallback for Jupyter / Interactive sessions
    json_path = Path.cwd().parents[2]
json_path = Path(json_path) / "tests" / "backend" / "json_files" / json_file_name
```

[6]:
```
# ## Set JSON reading properties
toolkit_api.load_properties_from_json(json_path)
```

[7]:
```
# ## Launch AEDT
thread_msg = toolkit_api.launch_thread(toolkit_api.launch_aedt)
idle = toolkit_api.wait_to_be_idle()
if not idle:
    print("AEDT not initialized.")
    sys.exit()
```

```
PyAEDT INFO: Python version 3.10.11 (tags/v3.10.11:7d4cc5a, Apr  5 2023, 00:38:17) [MSC␣
↪v.1929 64 bit (AMD64)].
PyAEDT INFO: PyAEDT version 0.26.1.
PyAEDT INFO: Initializing new Desktop session.
PyAEDT INFO: AEDT version 2025.2.
PyAEDT INFO: New AEDT session is starting on gRPC port 63649.
PyAEDT INFO: Starting new AEDT gRPC session on port 63649.
PyAEDT INFO: Launching AEDT server with gRPC transport mode: wnua
PyAEDT INFO: Electronics Desktop started on gRPC port 63649 after 10.3 seconds.
PyAEDT INFO: AEDT installation Path C:\Program Files\ANSYS Inc\v252\AnsysEM
PyAEDT INFO: Connected to AEDT gRPC session on port 63649.
PyAEDT WARNING: Service Pack is not detected. PyAEDT is currently connecting in Insecure␣
↪Mode.
PyAEDT WARNING: Please download and install latest Service Pack to use connect to AEDT␣
↪in Secure Mode.
PyAEDT INFO: Desktop has been released.
```

```
INFO - AEDT is released.
```

[8]:
```
# ## Create the model
setup = toolkit_api.create_model()
```

```
PyAEDT INFO: Python version 3.10.11 (tags/v3.10.11:7d4cc5a, Apr  5 2023, 00:38:17) [MSC␣
↪v.1929 64 bit (AMD64)].
PyAEDT INFO: PyAEDT version 0.26.1.
PyAEDT INFO: Initializing Desktop session.
PyAEDT INFO: AEDT version 2025.2.
PyAEDT INFO: AEDT installation Path C:\Program Files\ANSYS Inc\v252\AnsysEM
PyAEDT INFO: Connected to AEDT gRPC session on port 63649.
PyAEDT WARNING: Service Pack is not detected. PyAEDT is currently connecting in Insecure␣
↪Mode.
PyAEDT WARNING: Please download and install latest Service Pack to use connect to AEDT␣
↪in Secure Mode.
PyAEDT INFO: Python version 3.10.11 (tags/v3.10.11:7d4cc5a, Apr  5 2023, 00:38:17) [MSC␣
↪v.1929 64 bit (AMD64)].
PyAEDT INFO: PyAEDT version 0.26.1.
PyAEDT INFO: Returning found Desktop session with PID 5548!
PyAEDT INFO: Project Project2 has been created.
PyAEDT INFO: Added design 'Maxwell3D_34HOM0' of type Maxwell 3D.
PyAEDT INFO: AEDT objects correctly read
PyAEDT INFO: Project Project2 Saved correctly
PyAEDT INFO: Active Design set to Maxwell3D_34HOM0
```

```
INFO - Updating internal properties.
INFO - Toolkit is connected to AEDT design.
```

```
PyAEDT INFO: Parsing C:\Users\ansys\AppData\Local\Temp\pytest-of-ansys\pytest-2365\
↪Project2.aedt.
PyAEDT INFO: File C:\Users\ansys\AppData\Local\Temp\pytest-of-ansys\pytest-2365\Project2.
↪aedt correctly loaded. Elapsed time: 0m 0sec
PyAEDT INFO: aedt file load time 0.01245570182800293
PyAEDT INFO: Modeler class has been initialized! Elapsed time: 0m 0sec
PyAEDT INFO: Materials class has been initialized! Elapsed time: 0m 0sec
PyAEDT INFO: Parsing design objects. This operation can take time
PyAEDT INFO: Refreshing bodies from Object Info
PyAEDT INFO: Bodies Info Refreshed Elapsed time: 0m 0sec
PyAEDT INFO: 3D Modeler objects parsed. Elapsed time: 0m 0sec
PyAEDT INFO: Parsing design objects. This operation can take time
PyAEDT INFO: Refreshing bodies from Object Info
PyAEDT INFO: Bodies Info Refreshed Elapsed time: 0m 0sec
PyAEDT INFO: 3D Modeler objects parsed. Elapsed time: 0m 0sec
PyAEDT INFO: Union of 20 objects has been executed.
PyAEDT INFO: Adding new material to the Project Library: 3C81_pyETK
PyAEDT INFO: Material has been added in Desktop.
PyAEDT INFO: Dataset Mu_3C81_pyETK doesn't exist.
PyAEDT INFO: Dataset $Mu_3C81_pyETK created successfully.
PyAEDT INFO: Parsing design objects. This operation can take time
PyAEDT INFO: Refreshing bodies from Object Info
PyAEDT INFO: Bodies Info Refreshed Elapsed time: 0m 0sec
PyAEDT INFO: 3D Modeler objects parsed. Elapsed time: 0m 0sec
PyAEDT INFO: Adding new material to the Project Library: Copper_pyETK
PyAEDT INFO: Material has been added in Desktop.
PyAEDT INFO: Parsing design objects. This operation can take time
```

(continues on next page)

(continued from previous page)

```
PyAEDT INFO: Refreshing bodies from Object Info
PyAEDT INFO: Bodies Info Refreshed Elapsed time: 0m 0sec
PyAEDT INFO: 3D Modeler objects parsed. Elapsed time: 0m 0sec
PyAEDT INFO: Deleted 1 Objects: Layer1_Section1_Separate1.
PyAEDT INFO: Parsing design objects. This operation can take time
PyAEDT INFO: Refreshing bodies from Object Info
PyAEDT INFO: Bodies Info Refreshed Elapsed time: 0m 0sec
PyAEDT INFO: 3D Modeler objects parsed. Elapsed time: 0m 0sec
PyAEDT INFO: Deleted 1 Objects: Layer1_1_Section1_Separate1.
PyAEDT INFO: Parsing design objects. This operation can take time
PyAEDT INFO: Refreshing bodies from Object Info
PyAEDT INFO: Bodies Info Refreshed Elapsed time: 0m 0sec
PyAEDT INFO: 3D Modeler objects parsed. Elapsed time: 0m 0sec
PyAEDT INFO: Deleted 1 Objects: Layer1_2_Section1_Separate1.
PyAEDT INFO: Parsing design objects. This operation can take time
PyAEDT INFO: Refreshing bodies from Object Info
PyAEDT INFO: Bodies Info Refreshed Elapsed time: 0m 0sec
PyAEDT INFO: 3D Modeler objects parsed. Elapsed time: 0m 0sec
PyAEDT INFO: Deleted 1 Objects: Layer1_3_Section1_Separate1.
PyAEDT INFO: Parsing design objects. This operation can take time
PyAEDT INFO: Refreshing bodies from Object Info
PyAEDT INFO: Bodies Info Refreshed Elapsed time: 0m 0sec
PyAEDT INFO: 3D Modeler objects parsed. Elapsed time: 0m 0sec
PyAEDT INFO: Deleted 1 Objects: Layer1_4_Section1_Separate1.
PyAEDT INFO: Parsing design objects. This operation can take time
PyAEDT INFO: Refreshing bodies from Object Info
PyAEDT INFO: Bodies Info Refreshed Elapsed time: 0m 0sec
PyAEDT INFO: 3D Modeler objects parsed. Elapsed time: 0m 0sec
PyAEDT INFO: Deleted 1 Objects: Layer1_5_Section1_Separate1.
PyAEDT INFO: Boundary Winding Layer_1 has been created.
PyAEDT INFO: Boundary CoilTerminal Layer1_Section1 has been created.
PyAEDT INFO: Boundary CoilTerminal Layer1_1_Section1 has been created.
PyAEDT INFO: Boundary CoilTerminal Layer1_2_Section1 has been created.
PyAEDT INFO: Boundary CoilTerminal Layer1_3_Section1 has been created.
PyAEDT INFO: Boundary CoilTerminal Layer1_4_Section1 has been created.
PyAEDT INFO: Boundary CoilTerminal Layer1_5_Section1 has been created.
PyAEDT INFO: Boundary Matrix Matrix1 has been created.
PyAEDT INFO: Adding new material to the Project Library: material_bobbin_pyETK
PyAEDT INFO: Material has been added in Desktop.
PyAEDT INFO: Mesh class has been initialized! Elapsed time: 0m 0sec
PyAEDT INFO: Mesh class has been initialized! Elapsed time: 0m 0sec
PyAEDT INFO: Python version 3.10.11 (tags/v3.10.11:7d4cc5a, Apr  5 2023, 00:38:17) [MSC␣
↪v.1929 64 bit (AMD64)].
PyAEDT INFO: PyAEDT version 0.26.1.
PyAEDT INFO: Returning found Desktop session with PID 5548!
PyAEDT INFO: No project is defined. Project Project2 exists and has been read.
PyAEDT INFO: Added design 'circuit_Maxwell3D_34HOM0' of type Maxwell Circuit.
PyAEDT INFO: AEDT objects correctly read
PyAEDT INFO: ModelerCircuit class has been initialized! Elapsed time: 0m 0sec
PyAEDT INFO: ModelerMaxwellCircuit class has been initialized!
PyAEDT WARNING: Property Va has not been edited.Check if readonly
PyAEDT WARNING: Property VFreq has not been edited.Check if readonly
```

(continues on next page)

(continued from previous page)

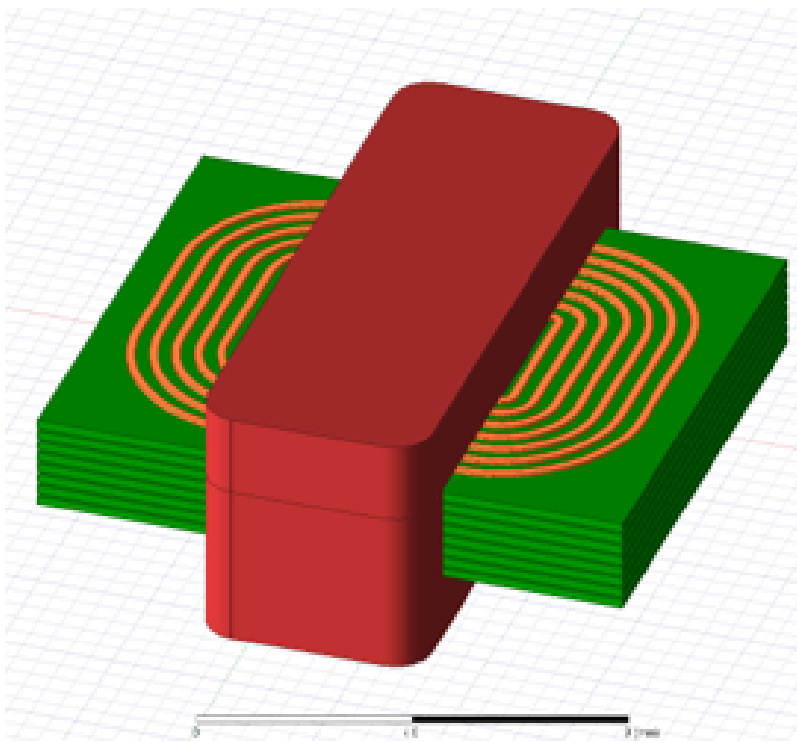
```
PyAEDT WARNING: Property R has not been edited.Check if readonly
PyAEDT INFO: Active Design set to 1;circuit_Maxwell3D_34HOM0
PyAEDT INFO: PostProcessor class has been initialized! Elapsed time: 0m 0sec
PyAEDT INFO: PostProcessor class has been initialized! Elapsed time: 0m 0sec
PyAEDT INFO: Post class has been initialized! Elapsed time: 0m 0sec
PyAEDT INFO: Active Design set to Maxwell3D_34HOM0
PyAEDT INFO: Active Design set to Maxwell3D_34HOM0
PyAEDT INFO: Active Design set to Maxwell3D_34HOM0
PyAEDT INFO: Active Design set to Maxwell3D_34HOM0
PyAEDT WARNING: No report category provided. Automatically identified AC Magnetic
PyAEDT INFO: Desktop has been released.
```

```
INFO - AEDT is released.
```

4.2 Create a transformer model in AEDT using the PyETK UI

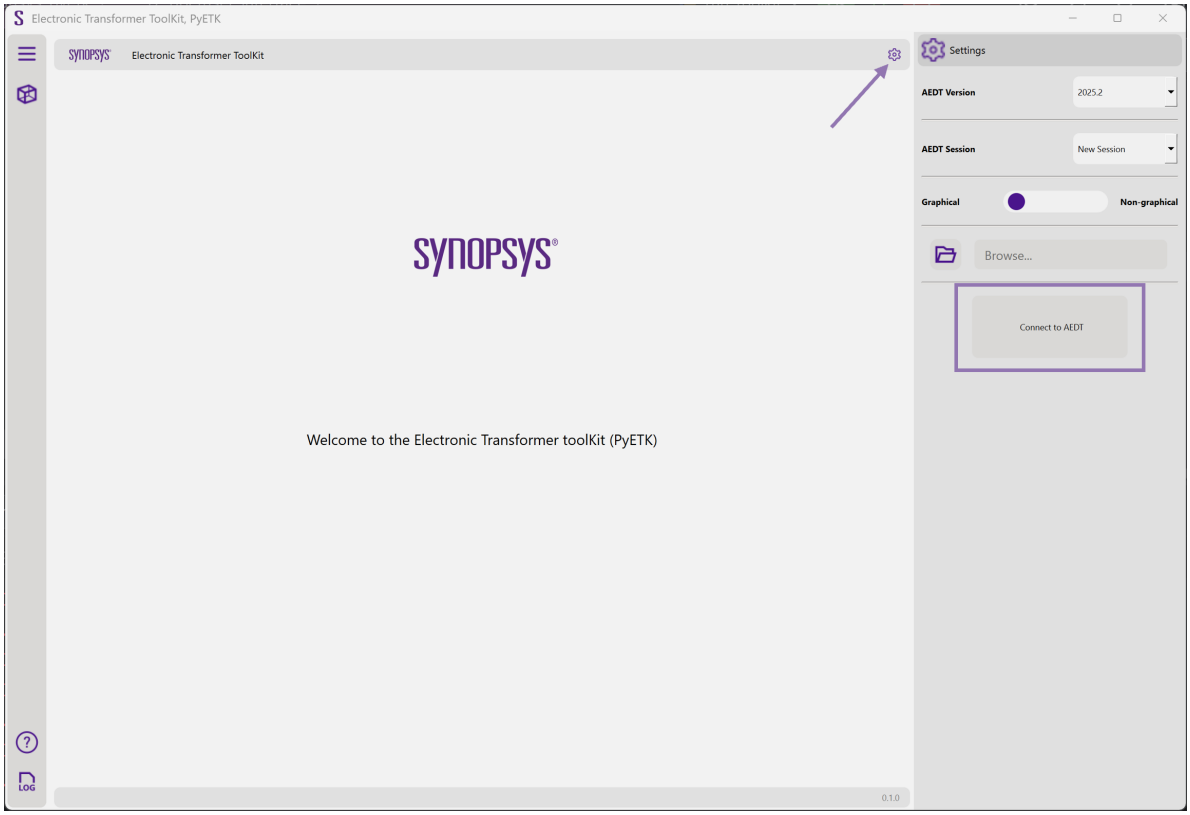
This example shows how to use the PyETK UI to create a transformer model in AEDT. It guides you through specifying settings and designing a transformer using the available fields and tools.

The design is for a planar transformer, which is commonly used in power electronics applications.



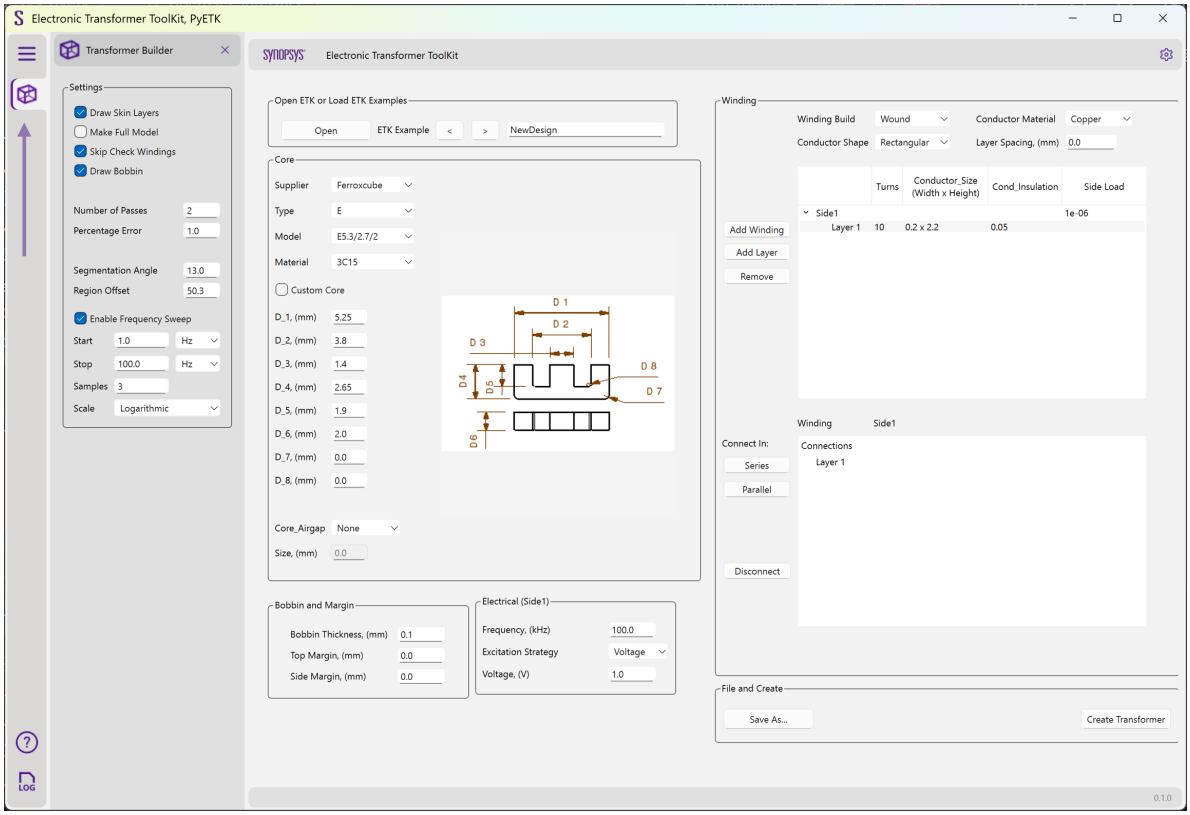
For simplicity, the example covers one submenu at a time, starting with the core model, dimensions, and materials. It then moves to the bobbin, winding, and excitation information and concludes with Maxwell settings.

1. Launch the PyETK UI and connect to AEDT by clicking **Connect to AEDT**. For PyETK installation instructions, see *Installation*.

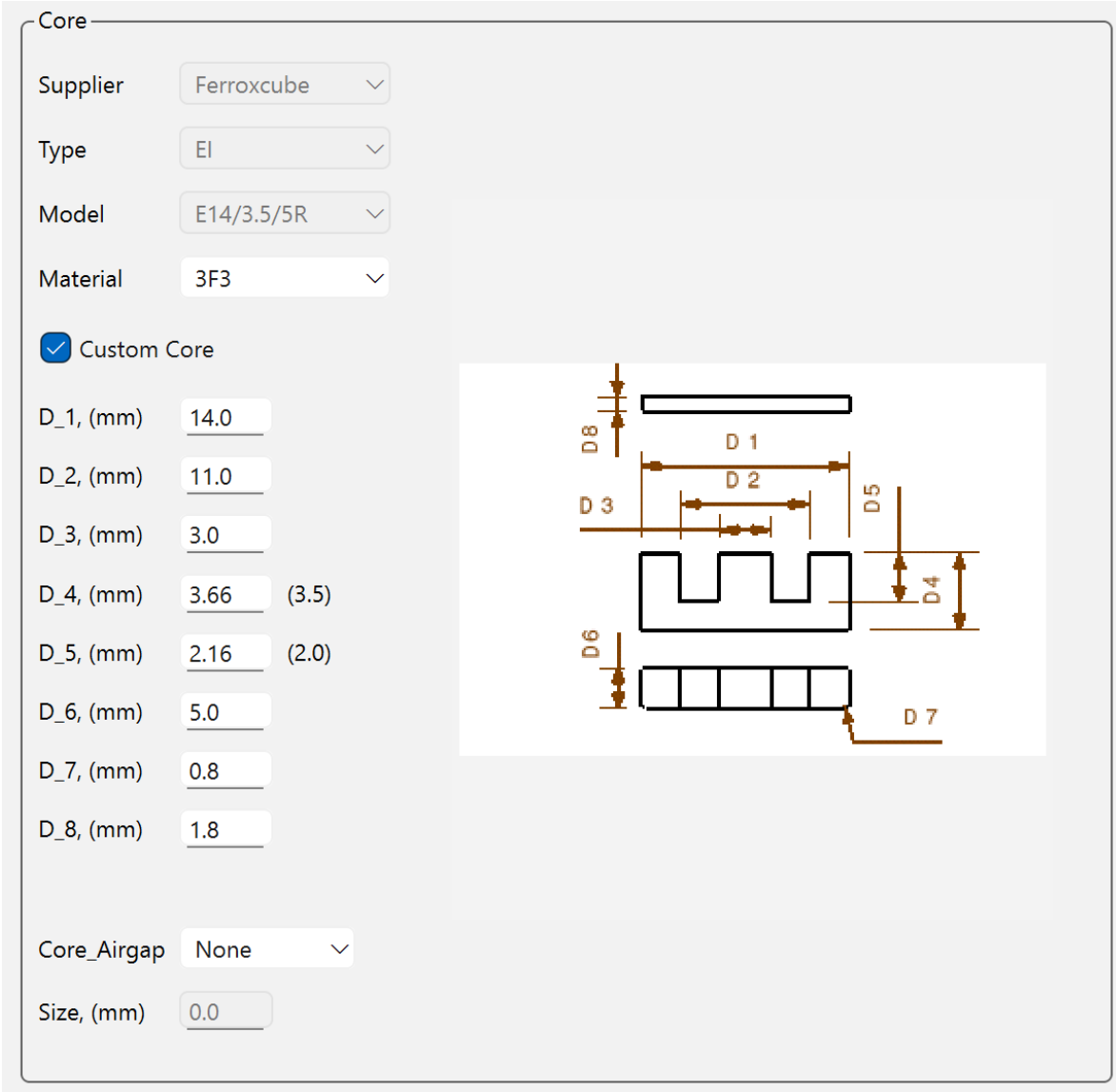


2. In the upper left corner of the UI, click the cube icon to display the **Transformer Builder** tab.

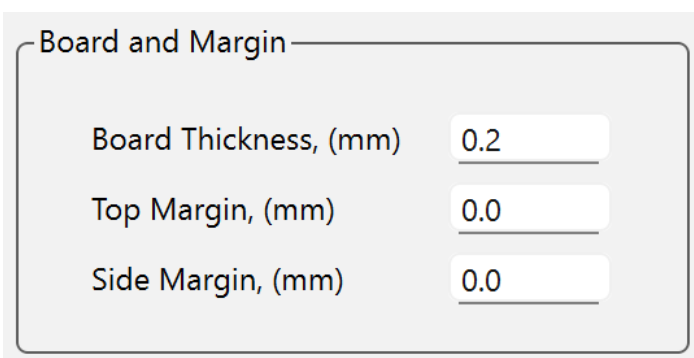
   This tab contains all the fields and tools to design and model electronic transformers.



3. In the **Core** area, select the core type, dimensions, and material. Select the **Custom Core** checkbox to apply a standard core shape with custom dimensions.



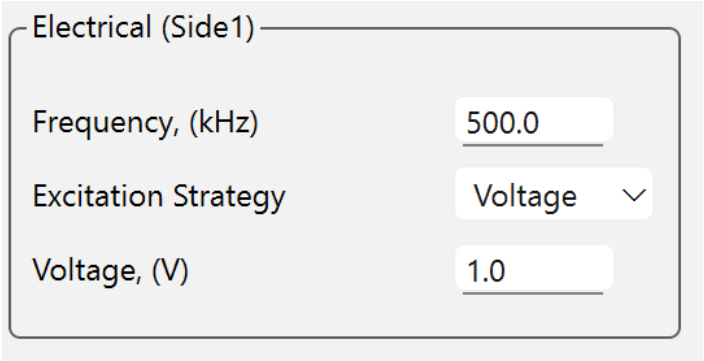
4. In the **Board and Margin** area, specify the thickness and margins in millimeters.



Note

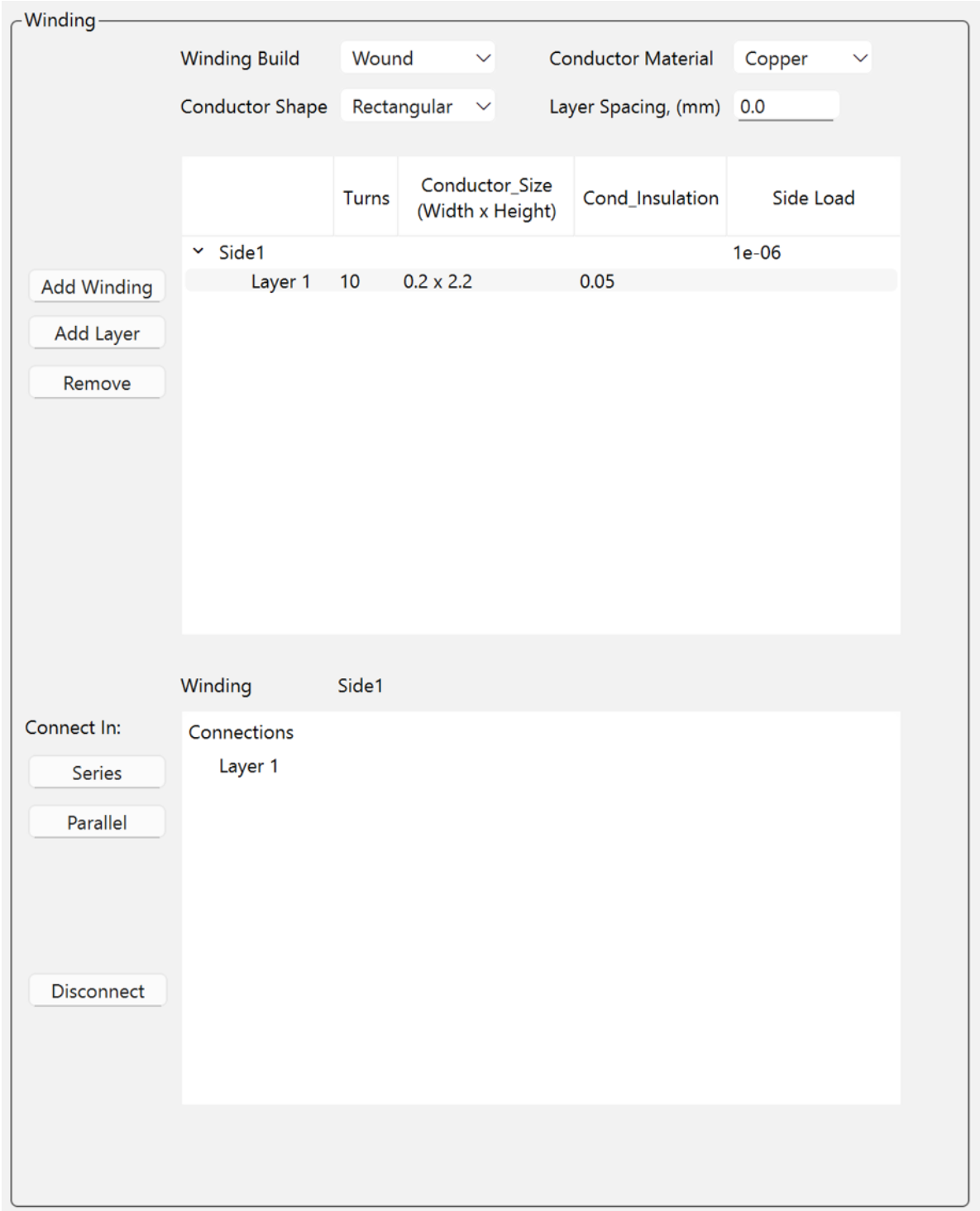
> In PyETK, the terms *board* and *bobbin* are interchangeable. *Board* refers to a planar build type, while *bobbin* refers to a wound build type. The fields in the UI are the same for both types.

5. In the **Electrical** area, modify the electrical source information, such as frequency, excitation strategy, and excitation value.



6. In the **Winding** area, create the transformer windings layer by layer by specifying the build type, material, number of turns, and turn spacing for each layer.

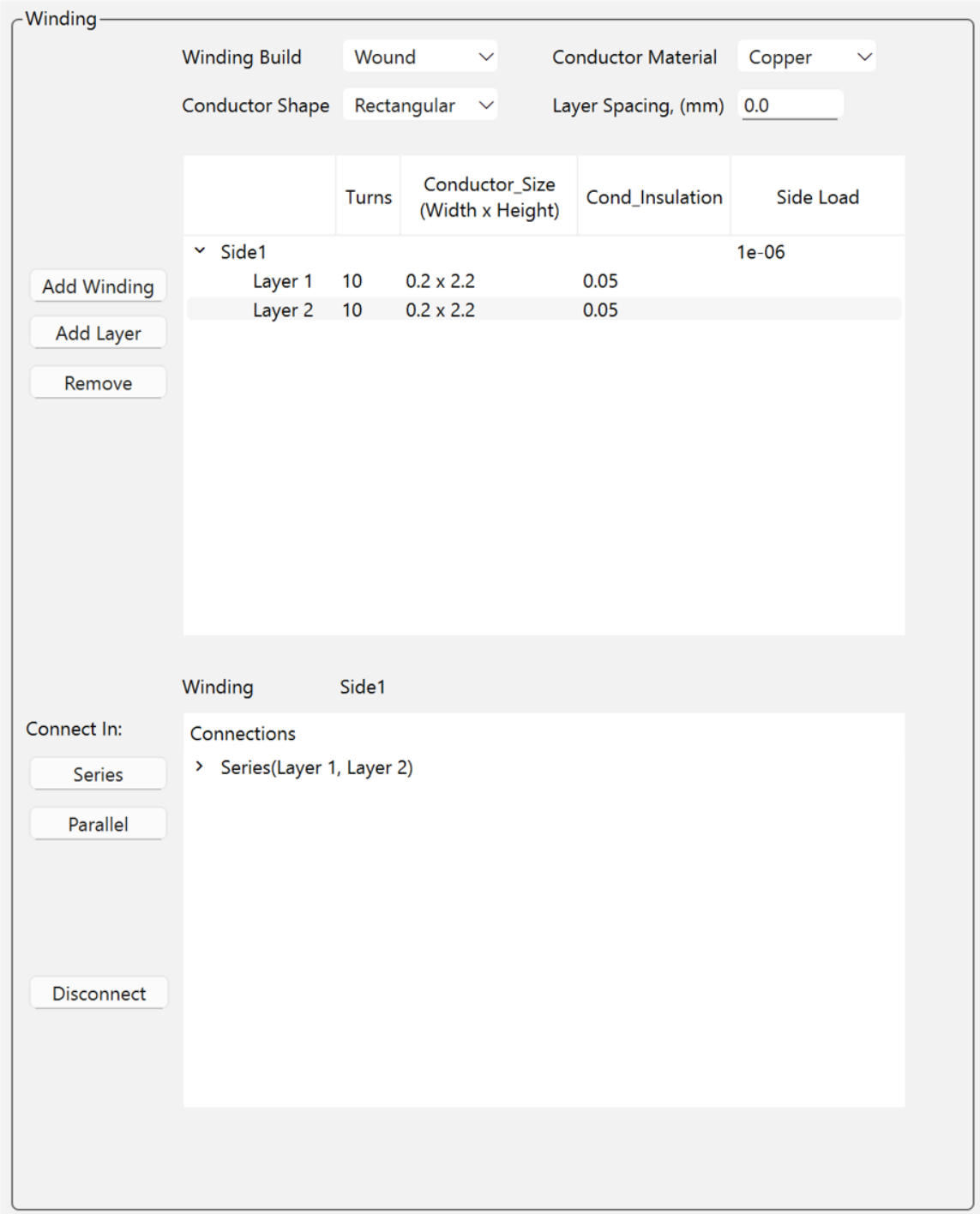
   The **Winding** area initializes as follows:



Important

The transformer requires at least one winding layer. The winding section initializes with one layer containing default values. You can modify these values or add more layers as needed. Additionally, you must specify the layer connection. By default, every layer in a winding side connects in series.

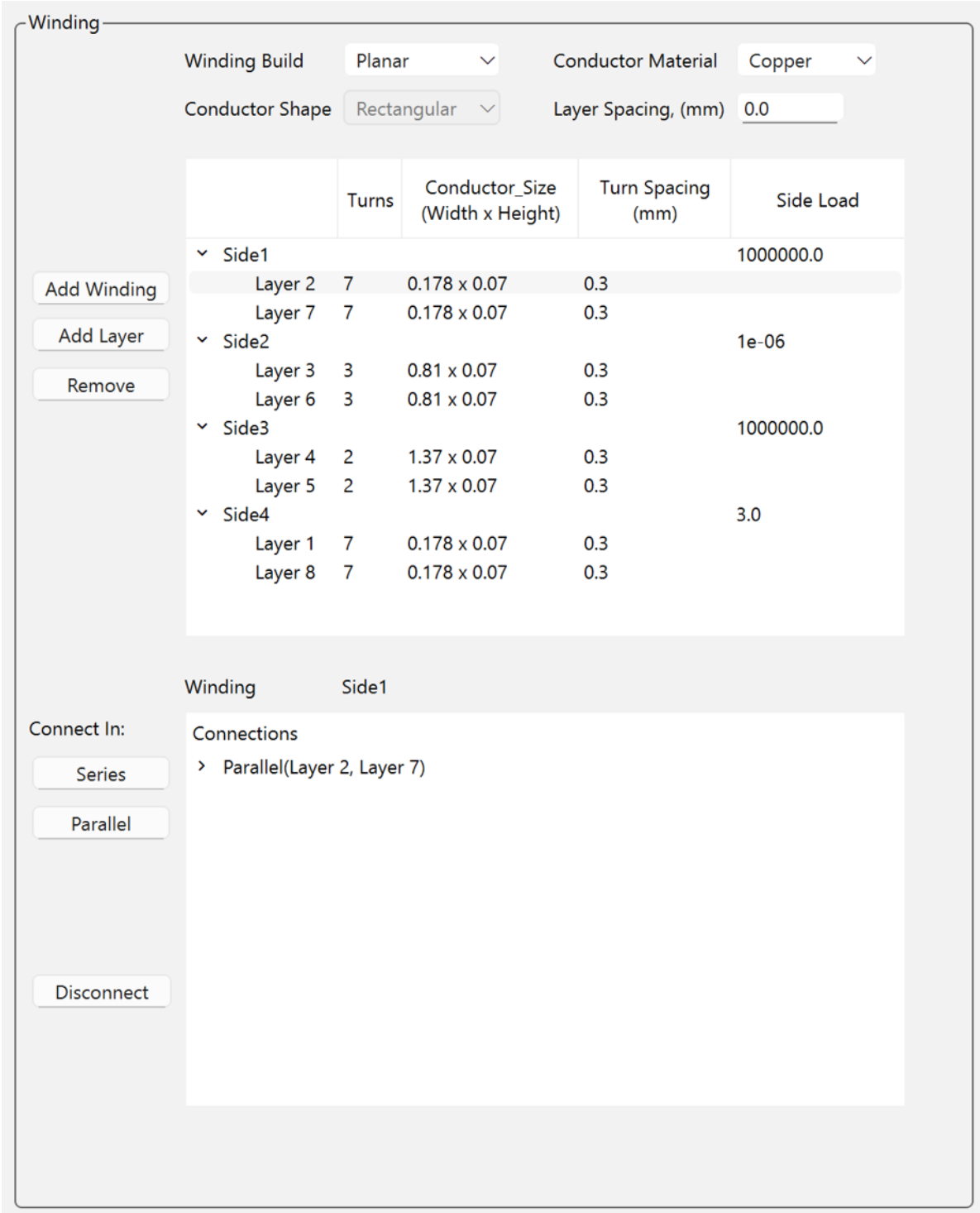
To modify the circuit connections between layers, select the layers of interest, click **Disconnect**, and then choose either **Series** or **Parallel** as the connection. You can also specify the connection between layers of each winding side by clicking **Side <number>** in the top Winding menu, and selecting either **Series** or **Parallel** to connect the layers in the bottom Winding/Connection menu.



After disconnecting, the default connection appears as follows:

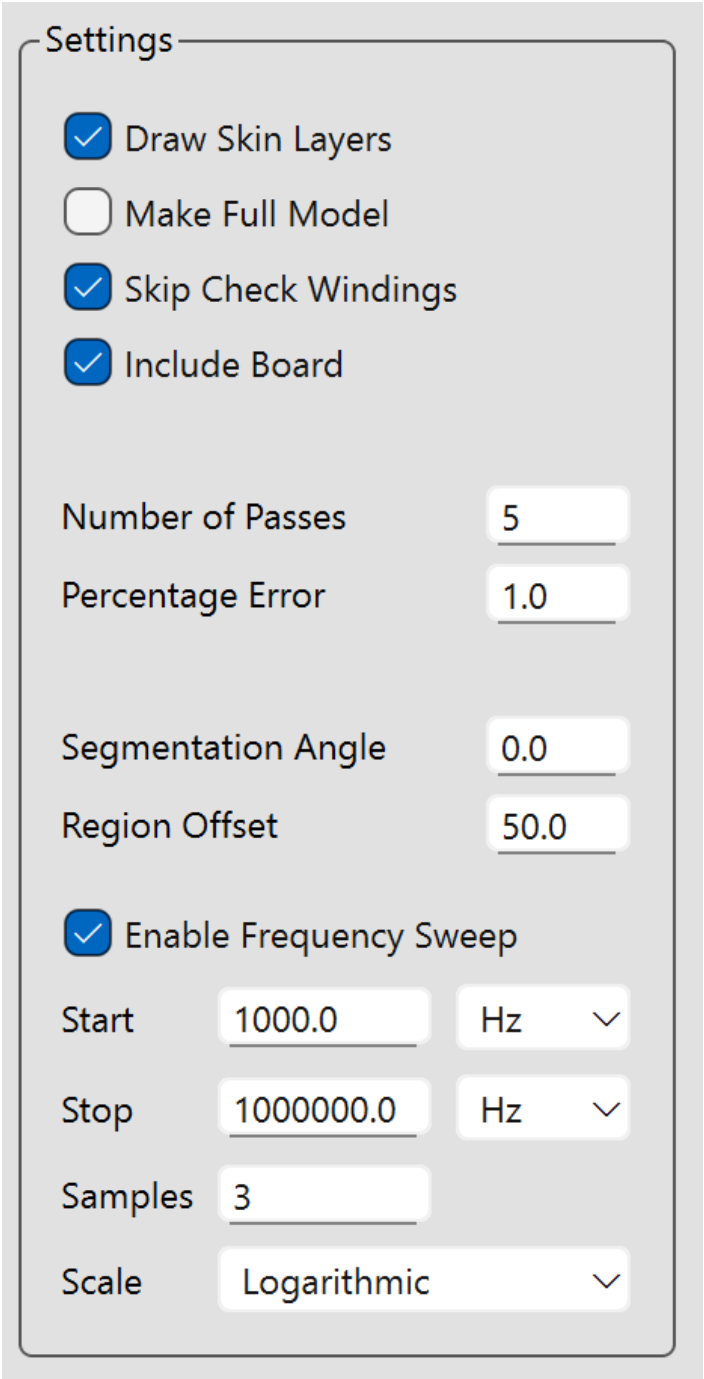
Connect the layers in parallel:

7. Create the winding layers and connections for the planar transformer as shown:

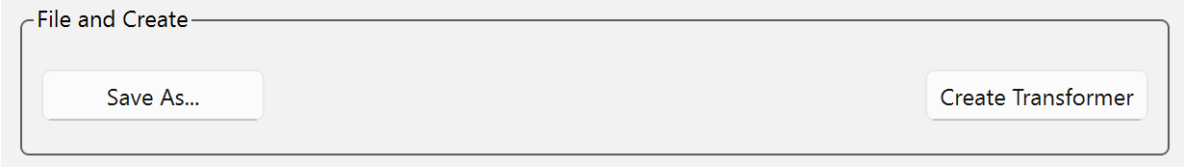


8. In the **Settings** pane on the left, specify Maxwell settings, including the number of passes, percentage error, maximum number of passes, and frequency sweeps.

   Maxwell settings control the accuracy and convergence of the Maxwell analysis performed after creating the transformer geometry in AEDT.



9. Click **Create Transformer** to create the transformer geometry in AEDT.



> **Note**
>
> Clicking **Save As** saves all information about the core, bobbin, winding, excitation, and Maxwell settings to a JSON file. You can later open this file and make modifications.

The log displays information about the creation process, including any errors or warnings.

4.2.1 Pre-packaged examples: Load a transformer model from a JSON file

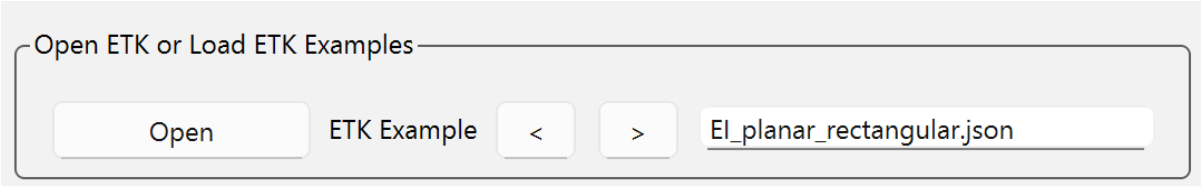
To enable third-party integration, PyETK uses a transformer definition stored in a versioned JSON configuration file. This file contains all the information, including sources, dimensions, materials, and advanced settings.

The JSON file can be used with the PyETK API, bypassing the UI.

To save the current transformer definition to a JSON file, click **Save As** in the UI. You can also create the JSON file manually by following the structure of an existing JSON file.

You can open or load a pre-packaged example in the PyETK UI:

- Click **Open** and select the JSON file.
- Use the forward and backward buttons to navigate through the JSON files.



> **Note**
>
> If you used ACT ETK, you can load your ACT JSON file in the PyETK UI by clicking **Open** and selecting your ACT JSON file. PyETK automatically parses the ACT JSON file and populates the fields in the PyETK UI with the values from this file. Review the populated fields to ensure all information is correctly transferred. If you plan to reuse the configuration file, click **Save As** to save the information in the ACT JSON file to the latest working JSON format. For more information, see *Use the PyETK UI to migrate an ACT JSON file to a PyETK JSON file*.

4.3 Use the PyETK UI to migrate an ACT JSON file to a PyETK JSON file

This example shows how to migrate an existing ACT JSON file for ETK to a PyETK JSON file.

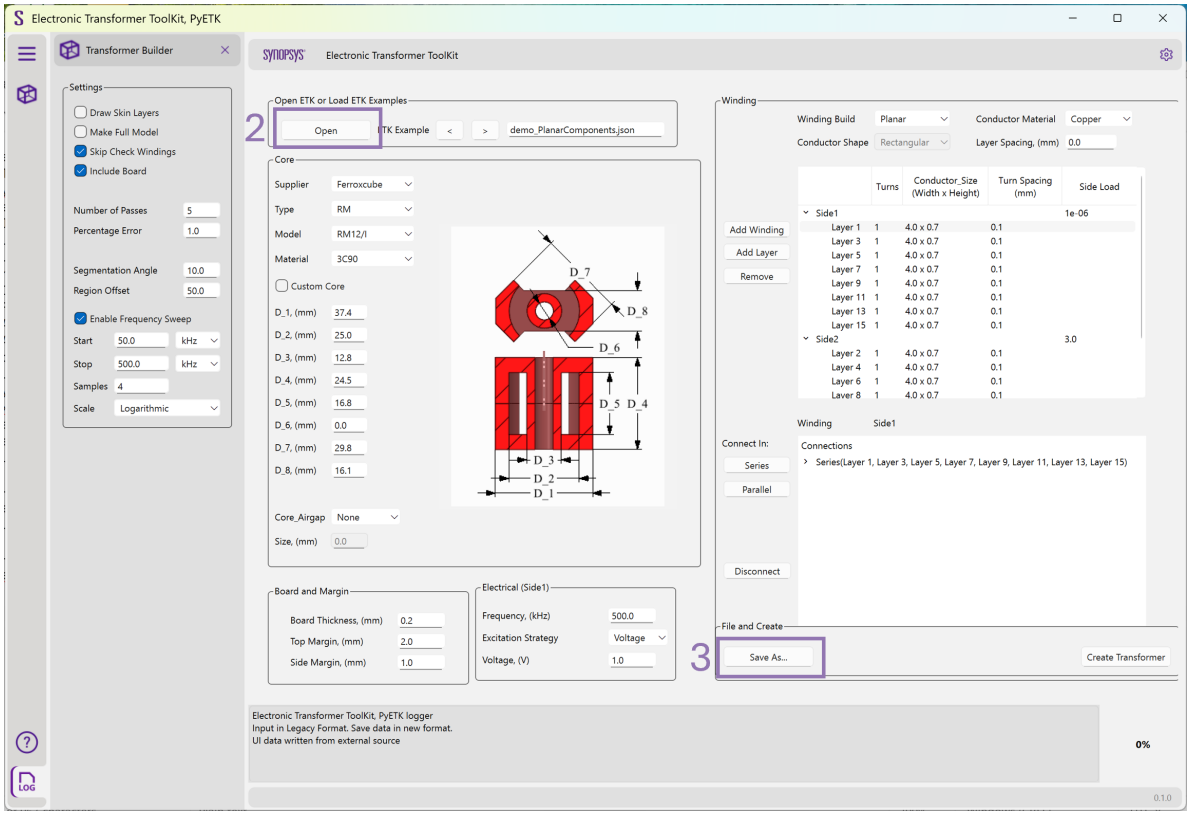
1. Verify that you have a valid ACT JSON file.

   This file contains the configuration and settings for your transformer. The ACT JSON file does not include a version. Its structure contains three main sections: `core_dimensions`, `winding_definitions`, and `setup_definitions`. It looks like this:

```
{
  "core_dimensions": {
    "segmentation_angle": 0,
    "supplier": "Ferroxcube",
    "core_type": "EI",
    "core_model": "E14/3.5/5R",
    "dimensions": {
    "D_1": "14.0",
    "D_2": "11.0",
    "D_3": "3.0",
    "D_4": "3.66",
    "D_5": "2.16",
    "D_6": "5.0",
    "D_7": "0.8",
    "D_8": "1.8"
    },
    "airgap": {
      "define_airgap": false
    }
  },
  "winding_definition": {
    "material_name": "Copper",
```

2. Open the PyETK UI and click the cube icon to display the **Transformer Builder** tab.
3. Click **Open** and select your ACT JSON file to load it into PyETK.

   PyETK automatically parses the ACT JSON file and populates the fields in the UI with the corresponding values from the ACT JSON file.
4. Review the populated fields in the UI to confirm that all the information transferred correctly.
5. If you want to reuse this configuration file, click **Save As** to save it in the latest working JSON format.



Note

The log provides information about the loading process, including any errors or warnings that may occur during the migration.

PyETKs versioned JSON file resembles the original ACT JSON configuration file. However, it has a more structured format, with clear sections for core dimensions, winding definitions, and setup definitions. This structure makes it easier to navigate and understand the different components of the transformer configuration.

```
{
  "json_version": "0.1.0",
  "core": {
    "supplier": "Ferroxcube",
    "type": "EI",
    "model": "E14/3.5/5R",
    "material": "3F3",
    "dimensions": {
      "D_1": 14.0,
      "D_2": 11.0,
      "D_3": 3.0,
      "D_4": 3.66,
      "D_5": 2.16,
      "D_6": 5.0,
      "D_7": 0.8,
      "D_8": 1.8
    },
    "airgap": {
      "define_airgap": false,
      "airgap_on_leg": "None",
      "airgap_value": 0.0
    }
  },
```

The version number in the PyETK JSON file helps to track changes and updates to the file format over time. It allows updates and improvements, including new features, without breaking compatibility with existing files.

CHAPTER

FIVE

RELEASE NOTES

This document contains the release notes for the project.

5.1 0.3.0 - April 17, 2026

Added

Included model validation	#73
General tidy up of UI inputs for QLineEdit and QTreeWidget.	#130

Documentation

Add example link in landing page	#97
Update main README.rst	#102
Add missing image of versioned json in documentation	#104
Overall review	#106
Use windows machine to build documentation	#115
Fix numbering in second example	#118
Add Ferroxcube material data link to help menu	#136

Fixed

Custom Core Dimensions	#109
Fix/logo	#131

Maintenance

Update CHANGELOG for v0.2.0	#94
Bump dev version into v0.3.dev0	#95
Update dependabot.yml configuration	#98
Run doc-build in self-hosted runner for examples	#99
Fix doc build for main and release workflows	#101
Add missing env variables to fix doc build	#111
Fix path to upload artifact	#112
Bump ansys actions to v10.2.9 to fix doc style	#113
Cleanup and use windows for doc-build jobs	#116
Fix generated artifacts in PR workflows	#117
Update PR workflow to rerun on title edit	#120
Add missing ansys license env var	#128

Miscellaneous

Rename mu(freq) and eps_r to match backend data model	#105

5.2 0.2.0 - March 03, 2026

Dependencies

Bump actions/checkout from 6.0.1 to 6.0.2	#51
Bump actions/setup-python from 6.1.0 to 6.2.0	#52
Bump ansys/actions from 10.1.4 to 10.2.5	#81

Documentation

Website Documentation	#82

Fixed

Edited the splash screen	#70
Remove Deletion of Top Level Parent Connection	#76
Remove AEDT Design title in toolkit Help Menu	#78
Deprecated arguments in generate_unique_project_name	#80
Delete Button added to delete row types Layers and Sides	#86

Maintenance

Update CHANGELOG for v0.1.0	#65
Bump version 0.2.0dev and update to year 2026	#66
Add new review changes	#71
Review of the backend code base	#85

Miscellaneous

Assign_matrix	#84
Changes related to UI / frontend logic	#88
Remove AEDT timeout	#93

5.3 0.1.0 - January 30, 2026

Added

JSON Example Files Added	#5
Settings Group Added to LHS UI Column	#186
Add Support for Bespoke and Reference Core Dimensions	#196
update json format	#210
BE tests improvement	#214
UI Save Button	#215
Add Connections to Data Exchange	#231
Add FE Tests	#232

Dependencies

Bump actions/setup-python from 6.0.0 to 6.1.0	#212
Bump softprops/action-gh-release from 2.3.3 to 2.5.0	#217
Bump actions/checkout from 5.0.0 to 6.0.1	#218
Bump actions/download-artifact from 5.0.0 to 7.0.0	#223
Bump actions/upload-artifact from 4.6.2 to 6.0.0	#224

Documentation

Remove Fixed Issues from README.rst	#11
Correct README.rst syntax	#13
Fix Help Menu Title PyETK Transformer Toolkit	#18
Improved backend API documentation	#33
Update badge link in README.rst	#44

Fixed

Fixed pyaedt arguments	#22
Backend model json_version typo	#49
Update backend properties, one property at a time	#54
Fixed layer spacing	#60
Rename create_transformer to create_model	#188
Fix load in side 1	#191
Fixes BE issues	#197
Frequency unit conversion	#205
Read from and Write gui_properties	#216
import fe_properties as properties	#229

Maintenance

Update license to Apache-2.0	#2
Clean doc and tests jobs	#8
Refactor into multiple workflors	#21
Update projects root level files	#24
Add missing checkout and wheelhouse	#56
Add missing job dependency in release job	#64
update CHANGELOG for v0.1.0a0	#178
bump 0.2.dev0	#179

Miscellaneous

Update Save As button text	#45
Update main_page.py	#46
PostProcessing and new HTTP request through /create_model	#184
remove/modify PyAEDT from toolkit naming convention	#189
Frontend Data and DataBase Manager	#228
Connections Definitions and Minor FE Fixes	#230

Test

Add additional bobbin unit tests	#6
Add additional data_manager unit tests	#7
Add circuit test	#9
Improved core tests	#14
UI Core with and without Airgap	#15
Add tests to FE data manager	#16

PYTHON MODULE INDEX

a

ansys.aedt.toolkits.electronic_transformer.backend.api, ??